mahanliaghatmand / **ChatDoc-AI**

Public

ChatDoc-AI

Technical Report & Project Roadmap

A decoder-only Transformer built from scratch in TensorFlow/Keras
for medical dialogue generation, trained on ChatDoctor-HealthCareMagic.

python 3.10+ | tensorflow 2.x | license MIT | status educational

64.1M	4 x 8	25.70	52.5K
parameters	layers x heads	test perplexity	training pairs

prompt format / generation

```
# training sequence layout
<|patient|> {question} <|doctor|> {answer}<|endoftext|>

$ streamlit run app.py
> loading medical_transformer.keras  (64,122,961 params)
>>> generate("I have headache. what should i do?")
sampling: temperature=0.8  top_k=40  max_new_tokens=100
```

Educational project. Not a medical device.
Outputs can be fluent but medically wrong. Never use for diagnosis or treatment.

Author: Mahan Liaghatmand

github.com/mahanliaghatmand/ChatDoc-AI

Report date: October 2026 | License: MIT

Table of Contents

1. Executive Summary 3
2. Background & Motivation 4
3. Repository Overview 4
4. Dataset & Preprocessing 5
5. Model Architecture 6
6. Training Methodology 8
7. Evaluation Results 9
8. Inference & Decoding 10
9. Application Layer 11
10. Technical Review & Findings 11
11. Limitations, Risks & Responsible Use 12
12. Roadmap 13
13. Reproducibility Guide 15
14. References & Glossary 16

1. Executive Summary

ChatDoc-AI is an end-to-end, from-scratch implementation of a GPT-style language model that answers patient questions in the voice of an online doctor.

The project implements every core component of a decoder-only Transformer with low-level Keras layers (sinusoidal positional encoding, causal multi-head self-attention, residual feed-forward blocks) and trains it on a 75,000-conversation subset of the public **ChatDoctor-HealthCareMagic-100k** dataset. The repository ships the training notebook, a Streamlit chat application, and the trained artifacts packaged for GitHub.

Aspect	Summary
Goal	Learn and demonstrate how a Transformer language model works internally, applied to medical Q&A.
Model	Decoder-only Transformer, 4 layers, 8 heads, d_{model} 512, FFN 2048, 64,122,961 parameters.
Data	74,995 valid patient/doctor pairs; split 52,496 train / 11,249 validation / 11,250 test.
Training	10 epochs, Adam (3e-4, clipnorm 1.0), batch 16; about 1,240 s per epoch on a Colab GPU (about 3.4 h).
Headline result	Test perplexity **25.70**, token accuracy **39.79%**, ROUGE-L 0.159, BERTScore F1 0.840 (50 samples).
Key finding	The model learned the *style* of doctor replies but not reliable medical *content*; it is still under-trained.
Status	Training pipeline complete and sound. The Streamlit app needs a fix before it can load the model (Section 10).

> **Safety notice**
> This system is an educational prototype. It produces fluent text that can be clinically wrong, repetitive or invented. It must not be used for diagnosis, treatment or any real medical decision.

Key takeaways

- **The engineering is solid.** Correct causal masking, padding-aware loss and accuracy, fixed seeds, a clean 70/15/15 split and multi-metric evaluation.
- **The model is under-trained, not over-fitted.** Validation loss was still falling at epoch 10 and early stopping never fired; the train/validation gap is only 0.125 nats.
- **Perplexity is a respectable baseline** for 64M parameters trained from scratch on about 52K examples, but generated answers remain unreliable.
- **The biggest blocker is deployment.** `app.py` does not yet register the custom layers and uses an encoder-decoder decoding loop that does not match this decoder-only model.
- **The roadmap** (Section 12) prioritises: fix the app, clean the data, train longer with a better schedule, evaluate rigorously, then deploy.

2. Background & Motivation

Why a small, from-scratch medical dialogue model is a useful learning project.

Large language models are usually consumed through pretrained checkpoints. That is efficient, but it hides how the model actually works. Building the network layer by layer exposes the mechanics that matter: how token and position information are combined, how a causal mask enforces left-to-right prediction, how residual connections and layer normalisation stabilise deep stacks, and how a padding-aware loss avoids learning from filler tokens.

Medical dialogue is a demanding application domain. The vocabulary is specialised, answers are long and structured, and the cost of a wrong answer is high. That makes it an excellent stress test, and a good place to practise honest evaluation and responsible communication of limitations.

Objectives

#	Objective	Status
O1	Implement a Transformer language model from scratch (no pretrained weights).	**Done**
O2	Train it on real doctor-patient conversations with correct padding-aware loss.	**Done**
O3	Evaluate with perplexity, token accuracy, ROUGE and BERTScore.	**Done (small sample)**
O4	Serve the model through an interactive chat interface.	**In progress**
O5	Document, package and publish the project professionally on GitHub.	**In progress**

Scope

- **In scope:** English text, single-turn Q&A, 255-token context, offline training on Colab, Streamlit demo.
- **Out of scope:** multi-turn memory, retrieval of medical sources, clinical validation, multilingual support, production deployment or regulatory compliance.

3. Repository Overview

What is in the GitHub repository and how the pieces fit together.

Path	Purpose
`Jupyter_Notebook.ipynb`	Complete pipeline: install, load data, tokenize, define model, train, evaluate, generate, save artifacts (24 cells, 3 code-heavy sections).
`app.py`	Streamlit chat UI (about 120 lines). Rebuilds the model from chunks, loads tokenizer and config, runs generation.
`artifacts/`	Saved model parts, tokenizer files and config.json produced by the notebook.
`requirements.txt`	Python dependencies for the app environment.
`LICENSE`	MIT License, copyright 2026 Mahan Liaghatmand.

Artifact packaging strategy

The trained `.keras` file is roughly 245 MB, above GitHub's 100 MB per-file limit. The project therefore stores the model as ordered binary chunks (`part_000, part_001, ...`) and `rebuild_model()` in `app.py` concatenates them on first launch. This keeps the repository cloneable without Git LFS at the cost of a one-time rebuild step.

```
artifacts/
  config.json              hyperparameters used for training
  tokenizer/               GPT-2 tokenizer files (vocab, merges, config)
  model_parts/part_000 ... ordered chunks, concatenated at runtime
  medical_transformer.keras   (generated on first run)
```

4. Dataset & Preprocessing

Source data, selection, formatting and the train/validation/test protocol.

Property	Value
Dataset	`lavita/ChatDoctor-HealthCareMagic-100k` (Hugging Face)
Origin	Patient questions and doctor replies from the HealthCareMagic platform
Full size	112,165 conversations (about 70.5 MB download)
Rows used	First 75,000, 74,995 remain after removing empty pairs
Fields	`input` (patient) and `output` (doctor)

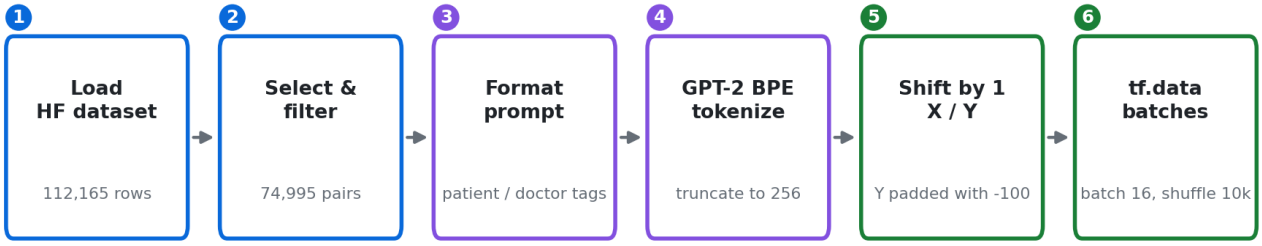


Figure 1. Data preparation pipeline from raw dataset to training batches.

Prompt serialisation

Each pair becomes one sequence so that a single next-token objective teaches the model both the question distribution and how a doctor answers:

```
<|patient|> {patient text} <|doctor|> {doctor text}<|endoftext|>
```

Inputs are the sequence without its last token; labels are the sequence without its first token. Sequences are truncated to 256 tokens, so model inputs are 255 tokens long. Input padding uses the GPT-2 end-of-text id; label padding uses **-100**, which the custom loss masks out.

Split protocol

Split	Fraction	Samples	Use
Train	70%	52,496	Gradient updates (3,281 steps per epoch at batch 16)
Validation	15%	11,249	Early stopping, learning-rate scheduling, model selection
Test	15%	11,250	Final perplexity and accuracy; generation metrics use the first 50

> **Reproducibility**
> The split uses scikit-learn `train_test_split` with `random_state=42`, and Python, NumPy and TensorFlow seeds are all set to 42.

Data-quality observations

- The notebook applies no text cleaning. Raw answers include boilerplate openings (for example a greeting from the platform) that the model reproduces verbatim.
- Some references are low quality: a displayed test reference is entirely upper-case and partly incoherent. Such noise lowers ROUGE and can leak into generations.
- The loss is computed over the whole sequence, so the model also spends capacity predicting patient text, not only the doctor answer (see roadmap item P3-4).

5. Model Architecture

A compact GPT-style decoder implemented with custom Keras layers.

The network embeds token ids, adds fixed sinusoidal position vectors, applies dropout, passes the result through four identical Transformer blocks and projects to vocabulary logits. Each block uses **post-layer-norm** residual connections (normalise after the residual add), as in the original Transformer.

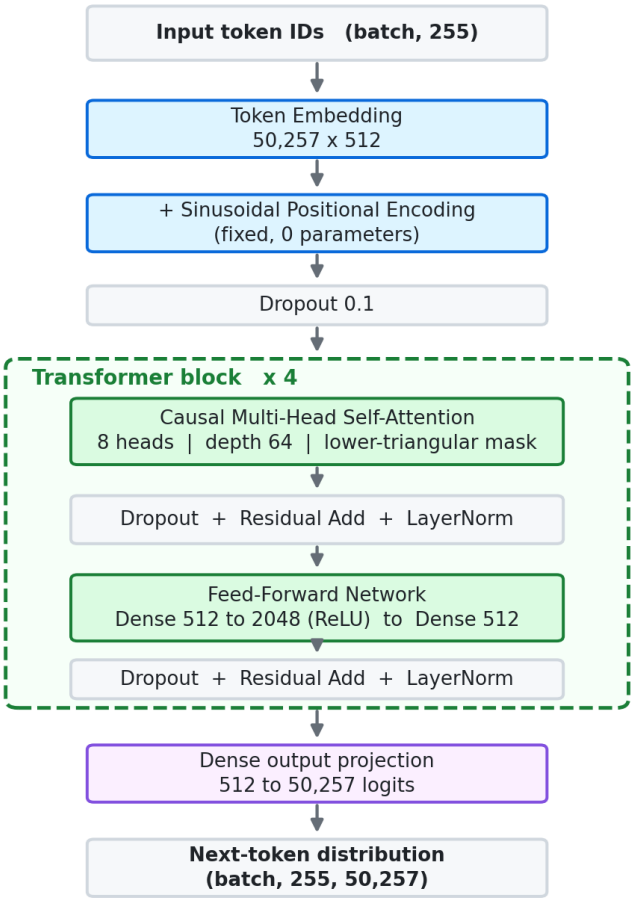


Figure 2. Forward pass of ChatDoc-AI. Shapes assume a batch dimension and a 255-token context.

Component	Configuration	Notes
Token embedding	50,257 x 512	GPT-2 BPE vocabulary
Positional encoding	Sinusoidal, length 255	Fixed, zero parameters, added to embeddings
Self-attention	8 heads, depth 64	Causal mask via lower-triangular matrix; scores scaled by 1/sqrt(64)
Feed-forward	512, 2048 (ReLU), 512	Applied position-wise
Normalisation	LayerNorm, eps 1e-6	Post-norm after each residual
Dropout	0.1	Embeddings, attention weights, sub-layer outputs
Output head	Dense 512 to 50,257	Separate weights (not tied to the embedding)

Parameter budget

Two vocabulary-sized matrices dominate the model: the embedding and the untied output projection hold about 80% of all parameters, while the four Transformer blocks contribute only about 20%. For such a small model this is an important design signal.

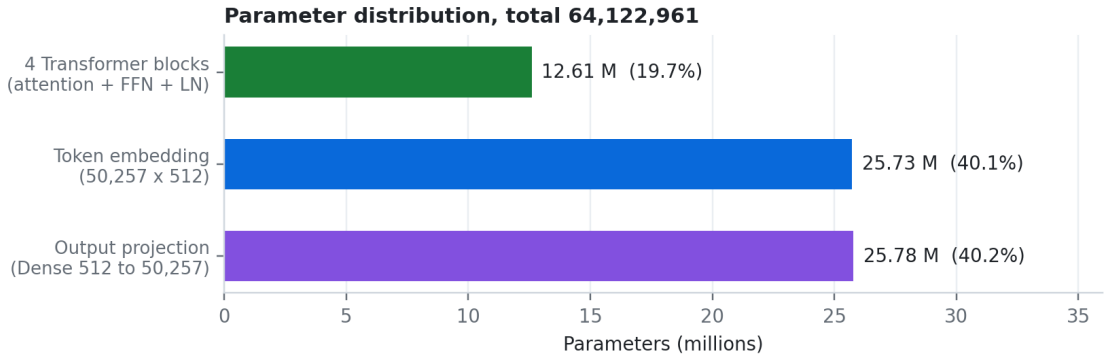


Figure 3. Parameter distribution. Each block holds 3,152,384 parameters (attention 1,050,624, FFN 2,099,712, LayerNorm 2,048).

Causal self-attention in detail

For input X, queries, keys and values are linear projections split into 8 heads. Attention weights are $\mathrm{softmax}(QK^T / \mathrm{sqrt}(d_k) + M)V$, where M is 0 on and below the diagonal and -1e9 above it, so position t can only attend to positions up to t. Because the mask is causal, real tokens never attend to later padding, which is why no separate padding mask is required.

6. Training Methodology

Objective, optimisation settings and the observed learning dynamics.

Training minimises masked sparse categorical cross-entropy on next-token prediction. The custom loss multiplies per-token loss by a mask that is zero wherever the label equals **-100**, then divides by the number of real tokens, so padding contributes neither loss nor accuracy.

Setting	Value
Optimiser	Adam, learning rate 3e-4, gradient clipping (clipnorm 1.0)
Batch size / epochs	16 / up to 10 (all 10 were run)
Callbacks	EarlyStopping (patience 2, restore best weights); ReduceLROnPlateau (factor 0.5, patience 1, min 1e-4)
Data pipeline	tf.data with shuffle buffer 10,000 and prefetch
Hardware / time	Google Colab GPU, about 371-379 ms per step, about 1,240 s per epoch, about 3.4 h total
Seeds	42 for Python, NumPy and TensorFlow

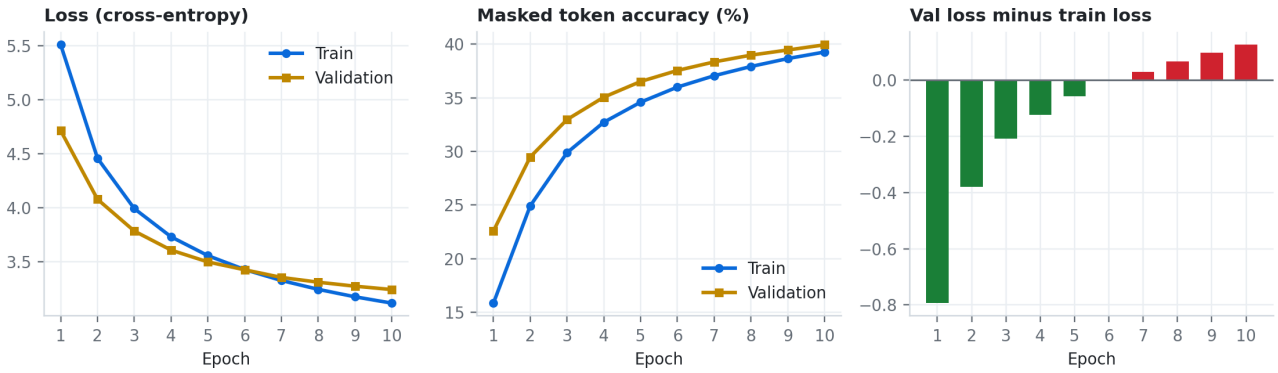


Figure 4. Training and validation loss, token accuracy, and the validation-minus-training loss gap per epoch.

Interpretation

- **Still learning at the end.** Validation loss fell every epoch (4.713 to 3.245) and was still decreasing by roughly 0.03 per epoch at epoch 10.
- **No overfitting.** The gap is negative for the first five epochs (dropout is active when training metrics are computed) and reaches only +0.125 at epoch 10.
- **Learning-rate schedule never triggered.** Because validation loss improved every epoch, ReduceLROnPlateau never reduced the rate; it stayed at 3e-4. Note that min_lr 1e-4 would limit any decay to a 3x range, so a warm-up plus cosine schedule is a better fit.
- **Early stopping never fired,** confirming the run ended because of the epoch limit, not convergence.

Epoch	Train loss	Val loss	Train acc	Val acc
1	5.5062	4.7128	15.85%	22.58%
2	4.4582	4.0790	24.92%	29.48%
3	3.9929	3.7854	29.88%	32.97%
4	3.7324	3.6095	32.72%	35.06%
5	3.5585	3.5002	34.59%	36.52%
6	3.4295	3.4277	36.00%	37.55%
7	3.3287	3.3570	37.06%	38.35%
8	3.2467	3.3128	37.92%	38.98%
9	3.1778	3.2760	38.66%	39.46%
10	3.1196	3.2448	39.27%	39.94%

7. Evaluation Results

Intrinsic language-model metrics and generation-quality metrics.

Test set (11,250 samples)

Metric	Value	Meaning
Test loss	3.2464	Mean cross-entropy per real token (nats)
Perplexity	**25.70**	exp(loss): effective branching factor of about 26 tokens
Token accuracy	**39.79%**	Share of next tokens predicted exactly (greedy argmax)

Generation quality (first 50 test samples, sampled replies)

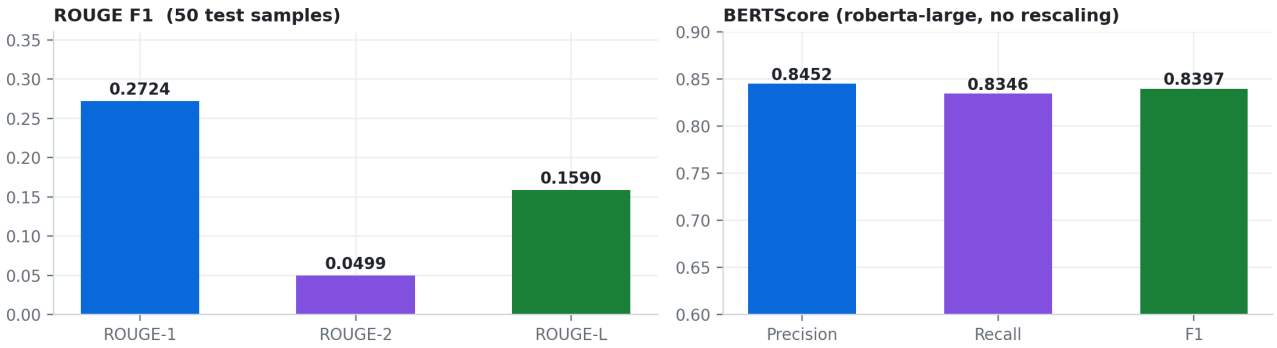


Figure 5. ROUGE F1 and BERTScore for 50 generated replies against reference doctor answers.

> **How to read these numbers**
> BERTScore was computed with roberta-large **without baseline rescaling**, so absolute values (about 0.84) are inflated; unrelated English text often scores 0.80+. Only 50 samples were evaluated and replies were sampled (temperature 0.8), so scores are noisy. None of these metrics measure medical correctness.

Qualitative analysis

Prompt	Observed behaviour	Assessment
"I have headache . what should i do ?"	Opens with a platform-style greeting, then claims migraine and nerve disorder, advises an ENT surgeon and an x-ray of the lumbosacral region, and repeats the word headache.	**Fluent but wrong**
Test patient about a toddler with asthma and allergies	Produces only a generic greeting and promise to give suggestions, then stops.	**Style only**

The model has clearly absorbed the register, greetings and vocabulary of the source doctors, but the clinical logic is not grounded. This is the expected failure mode of a small language model trained from scratch on noisy conversational data, and it is the main reason the project must stay educational.

8. Inference & Decoding

How replies are generated token by token.

Generation is autoregressive. The patient text is wrapped in the training prompt, tokenised, and the model predicts a distribution over the next token. A token is sampled, appended, and the process repeats until the end-of-text token or the length limit.

Parameter	Default	Effect
`temperature`	0.8	Lower values are safer and more repetitive; 0 means greedy argmax.
`top_k`	40	Sample only among the 40 most probable tokens.
`max_new_tokens`	100	Upper bound on reply length.
`Context window`	255	Only the most recent 255 tokens are kept for long prompts.

Performance note: each new token triggers a full forward pass over the padded 255-token window (no key/value cache), so latency grows linearly with reply length. The notebook wraps the forward pass in `tf.function` to reduce overhead.

9. Application Layer

The Streamlit chat interface.

`app.py` provides a chat-style UI with message history, a sidebar and a clear-chat button, and shows a prominent educational disclaimer. Model loading is wrapped in `st.cache_resource` so it happens once per server session.

Stage	Implementation
Startup	Rebuild `medical_transformer.keras` from `model_parts/part_*` if missing.
Loading	Read `config.json`, load tokenizer, load Keras model with `compile=False`.
Chat loop	Append user message, generate reply under a spinner, render and store in session state.
UX	Medical disclaimer caption, About sidebar, session-scoped history.

Blocking issues in the current app.py
The app cannot yet run the notebook's model. See Section 10 for the exact problems and fixes.

10. Technical Review & Findings

Code audit of the notebook and application, ordered by severity.

ID	Severity	Finding	Recommended fix
F1	**Critical**	`CUSTOM_OBJECTS` is empty, but the saved model contains `AddPositionalEncoding`, `CausalSelfAttention` and `TransformerBlock`; loading fails.	Copy the three classes into the app and register them in `custom_objects`.
F2	**Critical**	`generate()` assumes an encoder-decoder (two inputs, `model.predict([enc, dec])`). The model is decoder-only with one input.	Port `generate_reply` from the notebook: prompt format, single input, top-k sampling, EOS stop.
F3	**High**	Tokenizer folder is saved as `tokenizer` but the app reads `Tokenizer`; fails on case-sensitive systems.	Use identical casing in notebook and app.
F4	**High**	Custom layers lack `get_config()` and serialisation registration.	Add `get_config` and `@register_keras_serializable`.
F5	**Medium**	Prompt markers `<|patient|>` and `<|doctor|>` are split into several ordinary sub-word tokens.	Add them as special tokens and resize the embedding.
F6	**Medium**	Loss covers the full sequence including the patient question.	Mask prompt tokens so only doctor tokens train the model (or compare both).
F7	**Medium**	Learning-rate plateau schedule never triggered; min_lr 1e-4 limits decay.	Use warm-up plus cosine decay.

ID	Severity	Finding	Recommended fix
F8	**Medium**	Evaluation uses 50 sampled replies; BERTScore not baseline-rescaled.	Evaluate 500+ samples with greedy and sampled decoding; enable baseline rescaling.
F9	**Low**	Embeddings are not scaled by sqrt(d_{model}); post-norm blocks; no weight tying.	Try scaling, pre-LN, GELU and tied embeddings (saves about 25.8M parameters).
F10	**Low**	No cleaning of ALL-CAPS, very short or boilerplate answers; EOS can be truncated on long samples.	Add a cleaning step and ensure EOS survives truncation.
F11	**Low**	No README, tests, pinned dependency versions or CI.	Add README, unit tests (mask, loss, positional encoding), pin versions.

What is done well

Causal masking is correct; the masked loss and accuracy are implemented carefully with a safe label substitution; gradients are clipped; seeds and splits are reproducible; and evaluation goes beyond loss to ROUGE and BERTScore.

11. Limitations, Risks & Responsible Use

Risk	Impact	Mitigation
Medical hallucination	Users may act on wrong advice	Persistent disclaimer, educational framing, no deployment as a health tool
Repetition and incoherence	Low trust, confusing text	Repetition penalty, top-p sampling, longer training
Dataset bias and noise	Platform-specific phrasing, inconsistent answers	Clean and filter data; document provenance
Privacy	Source data are real patient questions	Do not add private data; follow dataset terms
Over-trust in metrics	ROUGE and BERTScore ignore correctness	Expert human evaluation before any claim of quality
Short context	Long histories are truncated	Longer context length or summarisation

Responsible-use commitments recommended for the repository: keep the disclaimer in the README and the UI, never present outputs as medical advice, state the dataset license and origin, and avoid hosting the demo in a context where users may mistake it for a clinical service.

12. Roadmap

A phased plan from a working prototype to a documented, evaluated and deployed project.

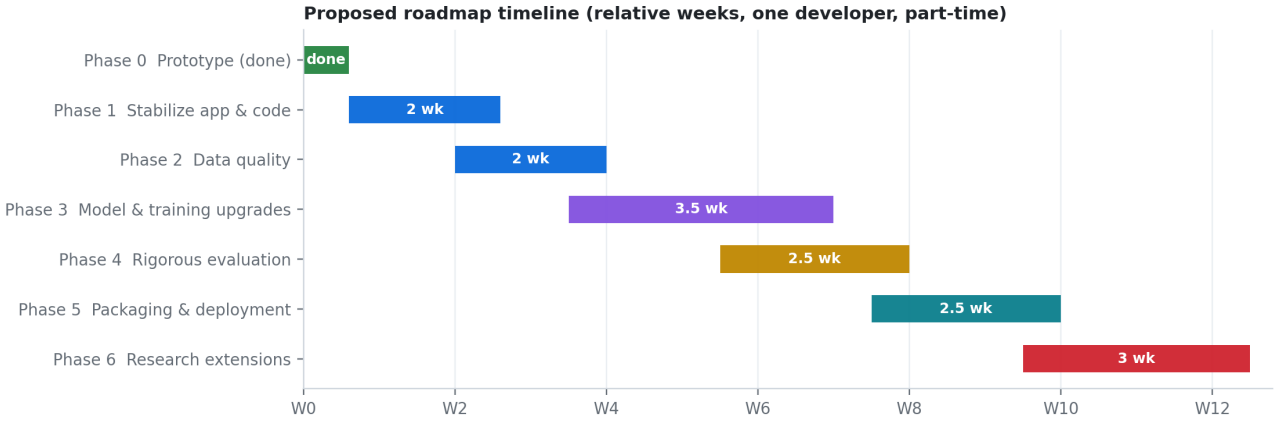


Figure 6. Proposed timeline. Phases overlap; durations are estimates for one developer working part-time.

Phase 1: Stabilize	Make the app run and match the notebook (about 2 weeks)	
ID	**Task**	**Deliverable / acceptance criterion**
P1-1	Register custom layers, add get_config	Model reloads from .keras in a fresh process
P1-2	Port decoder-only generation to app.py	App reply matches notebook output for a fixed seed
P1-3	Unify tokenizer path casing; pin requirements	Clean clone runs `streamlit run app.py` on Linux
P1-4	Add README and report to the repository	README and PDF committed

Phase 2: Data quality	Improve what the model learns from (about 2 weeks)	
ID	**Task**	**Deliverable / acceptance criterion**
P2-1	Filter ALL-CAPS, very short and duplicate answers	Documented filtering rules and removed-row counts
P2-2	Normalise boilerplate greetings and sign-offs	Reduced verbatim platform phrases in outputs
P2-3	Use all 112K rows; keep EOS on truncation	Rebuilt splits with fixed seed

Phase 3: Model & training	Raise quality at similar scale (about 3.5 weeks)	
ID	**Task**	**Deliverable / acceptance criterion**
P3-1	Train 20-30 epochs with warm-up + cosine decay	Validation loss plateaus; perplexity below 22
P3-2	Add special tokens for patient/doctor markers	Tokenizer extended, embedding resized
P3-3	Embedding scaling, GELU, pre-LN, optional weight tying	Ablation table with perplexity per change
P3-4	Train on doctor tokens only (prompt masking)	Compare against full-sequence loss
P3-5	Mixed precision and larger batch via accumulation	Epoch time reduced

Phase 4: Evaluation	Make claims defensible (about 2.5 weeks)	
ID	**Task**	**Deliverable / acceptance criterion**
P4-1	Evaluate 500+ samples, greedy and sampled	Mean and standard deviation reported
P4-2	BERTScore with baseline rescaling; add distinct-n and repetition rate	Metric suite in notebook
P4-3	Small human review rubric (relevance, safety, correctness)	20-50 rated replies with inter-rater notes
P4-4	Compare with fine-tuned DistilGPT-2 / GPT-2 baseline	Side-by-side results table

Phase 5: Packaging & deploy	Make it easy to run and share (about 2.5 weeks)	
ID	**Task**	**Deliverable / acceptance criterion**
P5-1	KV-cache, top-p sampling, repetition penalty	At least 3x faster generation
P5-2	Dockerfile and GitHub Actions (lint, tests)	Green CI badge in README
P5-3	Deploy demo (Hugging Face Spaces) with disclaimer	Public URL linked in README
P5-4	Unit tests for mask, loss, positional encoding	Tests pass in CI

Phase 6: Research extensions	Optional directions (about 3 weeks)	
ID	**Task**	**Deliverable / acceptance criterion**
P6-1	Retrieval-augmented answers from trusted medical sources	Answers cite sources
P6-2	Multi-turn dialogue with conversation memory	Context carried across turns
P6-3	Safety layer: emergency detection and refusal templates	Red-flag test set passes
P6-4	Scale study: layers, width and data size	Scaling plot of perplexity

Success criteria for the next release

Dimension	Today	Target
App runs from a clean clone	No	Yes, on Linux, macOS and Windows
Test perplexity	25.70	Below 22
Evaluation sample size	50	500 or more, with confidence intervals
Repetition in replies	Frequent	Measurably reduced (repetition penalty, top-p)
Documentation	License only	README, report, tests, CI

13. Reproducibility Guide

Run the application

```
git clone https://github.com/mahanliaghatmand/ChatDoc-AI.git
cd ChatDoc-AI
python -m venv .venv && source .venv/bin/activate     # Windows: .venv\Scripts\activate
pip install -r requirements.txt
streamlit run app.py                                  # http://localhost:8501
```

Retrain the model (Google Colab)

- Open `Jupyter_Notebook.ipynb` in Colab and select a GPU runtime.
- Run all cells: install dependencies, mount Drive, download the dataset, build splits, train (about 3.4 h), evaluate and save.
- Artifacts are written to `MyDrive/MedicalChatbot/`: model, tokenizer and config. Copy them into `artifacts/`, splitting the model file into `part_*` chunks below 100 MB.
- Notebook environment: TensorFlow 2.21.0, GPT2TokenizerFast, rouge-score, bert-score, scikit-learn.

Hyperparameter reference

Key	Value	Key	Value
`NUM_SAMPLES`	75,000	`DROPOUT`	0.1
`MAX_LENGTH`	256	`BATCH_SIZE`	16
`DMODEL`	512	`EPOCHS`	10
`NUM_HEADS`	8	`LR`	3e-4
`FF_DIM`	2048	`PAD_LABEL`	-100
`NUM_LAYERS`	4	`NUM_EVAL / SEED`	50 / 42

14. References & Glossary

- Vaswani et al., *Attention Is All You Need*, NeurIPS 2017 (arXiv:1706.03762).
- Radford et al., *Language Models are Unsupervised Multitask Learners* (GPT-2), 2019.
- Li et al., *ChatDoctor: A Medical Chat Model Fine-tuned on a LLaMA Model using Medical Domain Knowledge*, 2023 (arXiv:2303.14070).
- Zhang et al., *BERTScore: Evaluating Text Generation with BERT*, ICLR 2020 (arXiv:1904.09675).
- Lin, *ROUGE: A Package for Automatic Evaluation of Summaries*, 2004.
- Dataset: `lavita/ChatDoctor-HealthCareMagic-100k` on Hugging Face.

Glossary

Term	Definition
Perplexity	Exponential of the mean cross-entropy; lower means the model is less surprised by the data.
Causal mask	Mask that blocks attention to future positions so training matches left-to-right generation.
Top-k sampling	Sampling only from the k most likely next tokens.
BPE	Byte-pair encoding, the sub-word tokenisation used by GPT-2.
KV-cache	Stored attention keys and values that avoid recomputation during generation.
Post-LN / Pre-LN	Layer normalisation placed after or before each residual sub-layer.

About this report

All figures and numbers are taken from the repository's notebook outputs (Jupyter_Notebook.ipynb) and source code (app.py). Roadmap durations and targets are proposals, not measurements.